

LangChain ChromaDB Vector Store - Complete API Reference

A comprehensive guide to the **Chroma** vector store integration in LangChain Python.

Table of Contents

1. [Introduction](#)
 2. [Installation](#)
 3. [Core Concepts](#)
 4. [Initialization](#)
 5. [Document Management](#)
 - [Add Documents](#)
 - [Add Documents with Metadata](#)
 - [Update Documents](#)
 - [Delete Documents](#)
 - [Delete All Documents](#)
 - [Clear Collection](#)
 6. [Search & Retrieval](#)
 - [Similarity Search](#)
 - [Hybrid Search](#)
 - [Filter Search](#)
 - [Vector Search with Filters](#)
 - [Get Documents by IDs](#)
 - [Get Documents with Metadata](#)
 - [Get All Documents](#)
 7. [Query & Retrieval API](#)
 8. [Collection Management](#)
 - [Create Collection](#)
 - [Describe Collection](#)
 - [Delete Collection](#)
 - [Get Collection Count](#)
 9. [Configuration & Parameters](#)
 10. [Persistence & In-Memory](#)
 11. [Advanced Usage & Tips](#)
 12. [Common Pitfalls](#)
-

Introduction

Chroma is an open-source embedding database for AI applications. It provides a fast, easy-to-use vector database that makes it simple to store and search over millions of vectors. When integrated with LangChain, the **Chroma** class provides a seamless interface between your embedding models and Chroma.

Key Benefits:

- ⚡ **Fast:** Sub-millisecond query latency
- 💾 **Persistent:** Save to disk for production use
- 🔍 **Scalable:** Handles millions of vectors efficiently
- 📦 **Simple:** Drop-in replacement for in-memory stores

Installation

```
# Using pip
pip install -qU langchain-chroma

# Using uv (recommended)
uv add langchain-chroma

# For async support
pip install -qU langchain-chroma[async]
```

Core Concepts

What is Chroma?

Chroma is a cloud-native embedding database that stores and searches over millions of vectors. It's designed for production use and provides:

- **Vector storage:** Native support for storing high-dimensional vectors
- **Similarity search:** Fast cosine similarity, Euclidean distance, and more
- **Filter support:** Search with metadata filters
- **Hybrid search:** Combine vector similarity with keyword search
- **Persistence:** Save your data to disk
- **REST API:** Built-in REST API for programmatic access

Chroma vs. In-Memory Vector Store

Feature	In-Memory (InMemoryVectorStore)	Chroma
Speed	Fastest (zero I/O)	Fast (sub-ms queries)
Persistence	× No	✅ Yes
Scalability	Limited by RAM	Scales to millions
Production Ready	× No	✅ Yes

Rule of thumb: Use **InMemoryVectorStore** for development/testing. Use **Chroma** for production.

Initialization

Basic Initialization

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

# Create embeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

# Initialize Chroma
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db", # Set for persistence, remove for in-
memory
)
```

Initialization Parameters

Parameter	Type	Required	Description
<code>collection_name</code>	<code>str</code>	✓	Name of the collection to use
<code>embedding_function</code>	<code>Embeddings</code>	✓	LangChain embedding model
<code>persist_directory</code>	<code>str</code>	×	Path for persistence (omit for in-memory)
<code>collection_metadata</code>	<code>dict</code>	×	Metadata for the collection
<code>embeddings_metadata</code>	<code>dict</code>	×	Metadata for embedding model

Advanced Initialization

```
# Initialize with custom metadata
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
    collection_metadata={"description": "My document collection"},
    embeddings_metadata={"title": "OpenAI"},
)

# Initialize with existing collection
vector_store = Chroma(
    collection_name="my_documents", # Must match existing collection
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)
```

Document Management

Add Documents

Basic Add

```
# Add a single document
doc = {
    "page_content": "This is a sample document.",
    "metadata": {"title": "Sample", "category": "tech"},
}
vector_store.add_documents([doc])

# Add multiple documents
documents = [
    {"page_content": "Document 1", "metadata": {"id": "1"}},
    {"page_content": "Document 2", "metadata": {"id": "2"}},
]
vector_store.add_documents(documents)
```

Add Documents with Metadata

```
# Add documents with rich metadata
documents = [
    {
        "page_content": "Python is a programming language.",
        "metadata": {
            "id": "1",
            "author": "John",
            "category": "programming",
            "source": "book1",
            "date_created": "2024-01-01",
        },
    },
    {
        "page_content": "JavaScript is a scripting language.",
        "metadata": {
            "id": "2",
            "author": "Jane",
            "category": "programming",
            "source": "book2",
            "date_created": "2024-01-02",
        },
    },
]
vector_store.add_documents(documents)
```

Add Using `from_documents` Class Method

```
# Create Chroma directly from documents
documents = [
    {"page_content": "Document 1", "metadata": {"id": "1"}},
    {"page_content": "Document 2", "metadata": {"id": "2"}},
]

vector_store = Chroma.from_documents(
    documents=documents,
    embedding=embeddings,
    collection_name="my_documents",
    persist_directory="./chroma_db",
)

# Create from text chunks
texts = ["This is text 1.", "This is text 2."]
metadata_list = [
    {"id": "1", "category": "a"},
    {"id": "2", "category": "b"},
]

vector_store = Chroma.from_texts(
    texts=texts,
    metadatas=metadata_list,
    embedding=embeddings,
    collection_name="my_documents",
    persist_directory="./chroma_db",
)
```

Update Documents

```
# Update a single document
doc = {
    "page_content": "This is the updated document.",
    "metadata": {"id": "1", "category": "tech"},
}
vector_store.update_documents([doc])

# Update multiple documents
documents = [
    {
        "page_content": "Updated document 1.",
        "metadata": {"id": "1", "category": "tech"},
    },
    {
        "page_content": "Updated document 2.",
        "metadata": {"id": "2", "category": "tech"},
    },
]
vector_store.update_documents(documents)
```

Delete Documents

```
# Delete a single document by ID
vector_store.delete(documents=[{"id": "1"}])

# Delete multiple documents by IDs
vector_store.delete(documents=[{"id": "1"}, {"id": "2"}])

# Delete all documents in the collection
vector_store.delete(documents=[]) # Empty list deletes all
```

Delete All Documents

```
# Delete all documents (same as passing empty list)
vector_store.delete(documents=[])

# Or manually
for doc in vector_store.get():
    vector_store.delete(documents=[{"id": doc.id}])
```

Clear Collection

```
# Clear the entire collection (same as deleting all documents)
vector_store.clear()
```

Search & Retrieval

Similarity Search

Basic Similarity Search

```
# Simple similarity search
results = vector_store.similarity_search("What is Python?")

# Get top-k results
results = vector_store.similarity_search("What is JavaScript?", k=5)
```

Similarity Search with Score

```
# Get results with similarity scores
results = vector_store.similarity_search_with_score("What is Python?", k=5)
```

```
# Access scores
for doc, score in results:
    print(f"Document: {doc.page_content}, Score: {score}")
# Output: Document: Python is a programming language., Score: 0.95
```

Similarity Search with Filter

```
# Search with metadata filter
results = vector_store.similarity_search(
    "What is JavaScript?",
    k=5,
    filter={"category": "programming"},
)
```

Hybrid Search

```
# Combine vector similarity with keyword search
results = vector_store.hybrid_search(
    "Python programming",
    k=5,
    alpha=0.7, # Weight: 0.7 for vector, 0.3 for keyword
)
```

Vector Search with Filters

```
# Search with multiple filters
results = vector_store.similarity_search(
    "What is JavaScript?",
    k=5,
    filter={
        "category": "programming",
        "author": "John",
    },
)

# Search with OR filter
results = vector_store.similarity_search(
    "What is Python?",
    k=5,
    filter={"category": {"$or": ["programming", "software"]}},
)

# Search with NOT filter
results = vector_store.similarity_search(
    "What is JavaScript?",
```

```
        k=5,
        filter={"author": {"$ne": "John"}},
    )

# Search with range filter
results = vector_store.similarity_search(
    "What is Python?",
    k=5,
    filter={"date_created": {"$gte": "2023-01-01", "$lte": "2024-01-01"}},
)
```

Get Documents by IDs

```
# Get documents by IDs (no search)
docs = vector_store.get(documents=[{"id": "1"}, {"id": "2"}])

# Get by ID with metadata
docs = vector_store.get(documents=[{"id": "1"}])

# Access document data
for doc in docs:
    print(f"ID: {doc.id}, Content: {doc.page_content}, "
          f"Metadata: {doc.metadata}")
```

Get Documents with Metadata

```
# Get documents with their metadata
docs = vector_store.get(documents=[{"id": "1"}, {"id": "2"}])

# Access metadata
for doc in docs:
    print(f"ID: {doc.id}, "
          f"Content: {doc.page_content}, "
          f"Metadata: {doc.metadata}")
```

Get All Documents

```
# Get all documents in the collection
docs = vector_store.get()

# Access all documents
for doc in docs:
    print(f"ID: {doc.id}, Content: {doc.page_content}")
```

Query & Retrieval API

add_documents()

```
def add_documents(documents: List[Document]) -> None
```

Add documents to the collection.

Parameters:

- **documents** (List[Document]): List of documents to add.

Example:

```
documents = [  
    {"page_content": "Hello world", "metadata": {"id": "1"}},  
    {"page_content": "Goodbye world", "metadata": {"id": "2"}},  
]  
vector_store.add_documents(documents)
```

add_texts()

```
def add_texts(  
    texts: List[str],  
    metadatas: Optional[List[dict]] = None,  
    batch_size: int = 100,  
) -> None
```

Add texts with optional metadata.

Parameters:

- **texts** (List[str]): List of text strings.
- **metadatas** (Optional[List[dict]]): List of metadata dicts.
- **batch_size** (int): Batch size for adding documents.

Example:

```
texts = ["Text 1", "Text 2"]  
metadatas = [{"id": "1"}, {"id": "2"}]  
vector_store.add_texts(texts, metadatas)
```

delete()

```
def delete(documents: List[Document]) -> None
```

Delete documents from the collection.

Parameters:

- **documents** (List[Document]): List of documents to delete. Empty list deletes all.

Example:

```
# Delete specific document
vector_store.delete(documents=[{"id": "1"}])

# Delete all documents
vector_store.delete(documents=[])
```

get()

```
def get(
    documents: Optional[List[Document]] = None,
    where: Optional[dict] = None,
    where_document: Optional[dict] = None,
    sort: Optional[dict] = None,
    limit: Optional[int] = None,
    offset: Optional[int] = None,
    fields: Optional[List[str]] = None,
) -> List[Document]
```

Get documents from the collection.

Parameters:

- **documents** (Optional[List[Document]]): Documents to retrieve. Empty list gets all.
- **where** (Optional[dict]): Filter by metadata.
- **where_document** (Optional[dict]): Filter by document content.
- **sort** (Optional[dict]): Sort documents.
- **limit** (Optional[int]): Maximum number of documents.
- **offset** (Optional[int]): Offset for pagination.
- **fields** (Optional[List[str]]): Fields to retrieve.

Example:

```
# Get all documents
docs = vector_store.get()
```

```
# Get documents by ID
docs = vector_store.get(documents=[{"id": "1"}])

# Filter by metadata
docs = vector_store.get(where={"category": "programming"})

# Get top-k similar documents
results = vector_store.similarity_search("What is Python?", k=5)

# Get documents with score
results = vector_store.similarity_search_with_score("What is Python?", k=5)
```

similarity_search()

```
def similarity_search(
    query: str,
    k: int = 4,
    filter: Optional[dict] = None,
) -> List[Document]
```

Perform similarity search.

Parameters:

- **query** (str): Search query.
- **k** (int): Number of results to return.
- **filter** (Optional[dict]): Metadata filter.

Example:

```
results = vector_store.similarity_search("What is Python?", k=5)
```

similarity_search_with_score()

```
def similarity_search_with_score(
    query: str,
    k: int = 4,
    filter: Optional[dict] = None,
) -> List[Document]
```

Perform similarity search with similarity scores.

Parameters:

- `query` (str): Search query.
- `k` (int): Number of results to return.
- `filter` (Optional[dict]): Metadata filter.

Example:

```
results = vector_store.similarity_search_with_score("What is Python?", k=5)
for doc, score in results:
    print(f"Score: {score:.4f}, Content: {doc.page_content}")
```

hybrid_search()

```
def hybrid_search(
    query: str,
    k: int = 4,
    filter: Optional[dict] = None,
    alpha: float = 0.5,
) -> List[Document]
```

Perform hybrid search combining vector and keyword search.

Parameters:

- `query` (str): Search query.
- `k` (int): Number of results to return.
- `filter` (Optional[dict]): Metadata filter.
- `alpha` (float): Weight for vector search (0 to 1).

Example:

```
results = vector_store.hybrid_search("Python programming", k=5, alpha=0.7)
```

Collection Management

Create Collection

```
# Initialize Chroma (creates collection if it doesn't exist)
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)
```

Describe Collection

```
# Get collection information
collection = vector_store._client.get_collection(name="my_documents")

# Access collection details
print(f"Name: {collection.name}")
print(f"Size: {collection.count()}")
print(f"Dimensions: {collection.arity}")
print(f"Metadata: {collection.metadata}")
```

Delete Collection

```
# Delete the collection
vector_store._client.delete_collection(name="my_documents")

# Or clear all documents
vector_store.clear()
```

Get Collection Count

```
# Get number of documents
count = vector_store._client.get_collection(name="my_documents").count()
print(f"Documents: {count}")
```

Configuration & Parameters

Chroma Constructor Parameters

Parameter	Type	Default	Description
<code>collection_name</code>	<code>str</code>	-	Name of the collection
<code>embedding_function</code>	<code>Embeddings</code>	-	LangChain embedding model
<code>persist_directory</code>	<code>str</code>	-	Path for persistence
<code>collection_metadata</code>	<code>dict</code>	-	Collection metadata
<code>embeddings_metadata</code>	<code>dict</code>	-	Embeddings metadata
<code>allow_resetting</code>	<code>bool</code>	<code>True</code>	Allow resetting collection
<code>chroma_client_kwargs</code>	<code>dict</code>	-	Chroma client kwargs
<code>chroma_persistent_kwargs</code>	<code>dict</code>	-	Chroma persistent kwargs

Filter Syntax

```
# Basic filters
filter = {"category": "programming"}

# Multiple values (OR)
filter = {"category": {"$in": ["python", "javascript"]}}

# Single value (NOT)
filter = {"author": {"$ne": "John"}}

# Range filters (numeric)
filter = {"score": {"$gte": 0.8, "$lte": 0.9}}

# Range filters (datetime)
filter = {"created_at": {"$gte": "2023-01-01", "$lte": "2024-01-01"}}

# Complex filters
filter = {
    "category": "programming",
    "author": {"$in": ["John", "Jane"]},
}

# OR filter
filter = {
    "category": {"$or": ["programming", "software"]},
}
```

Persistence & In-Memory

In-Memory (No Persistence)

```
# No persist_directory = in-memory (data lost on process restart)
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=embeddings,
    # persist_directory parameter omitted
)
```

Persistent (Save to Disk)

```
# With persist_directory = data survives process restart
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)
```

```
# Data stored in: ./chroma_db/chroma/  
# Structure:  
# chroma_db/chroma/  
# |— my_documents/  
# |   |— index  
# |   |— snapshot.json  
# |   |— ...
```

Persistence Considerations

1. **Performance:** Persistent Chroma is slightly slower than in-memory
2. **Storage:** Each document ~50-100 KB (vector + metadata)
3. **Index:** Chroma builds an index on startup (first query takes longer)
4. **Backup:** Copy the `chroma_db` directory for backups

Advanced Usage & Tips

Working with Large Collections

```
# For large collections, use batch operations  
batch_size = 1000  
for i in range(0, 10000, batch_size):  
    batch = documents[i:i + batch_size]  
    vector_store.add_documents(batch)
```

Custom Indexing

```
# Chroma automatically indexes on first query  
# Subsequent queries are fast  
results = vector_store.similarity_search("What is Python?", k=5)
```

Custom Metric

```
# Chroma supports multiple similarity metrics  
# Default: cosine similarity  
# Options: L2, dot_product, etc.  
# Set via chroma_persistent_kwargs  
  
vector_store = Chroma(  
    collection_name="my_documents",  
    embedding_function=embeddings,  
    persist_directory="./chroma_db",  
    chroma_persistent_kwargs={  
        "persist_directory": "./chroma_db",
```

```
        "allow_resetting": True,  
    },  
)
```

Async Support

```
from langchain_chroma import AsyncChroma  
from langchain_openai import OpenAIAsyncEmbeddings  
  
# Async Chroma  
async_vector_store = AsyncChroma(  
    collection_name="my_documents",  
    embedding_function=OpenAIAsyncEmbeddings(),  
    persist_directory="./chroma_db",  
)  
  
# Async search  
results = await async_vector_store.similarity_search("What is Python?",  
k=5)
```

Custom Metadata

```
# Store custom metadata  
documents = [  
    {  
        "page_content": "Python is great.",  
        "metadata": {  
            "id": "1",  
            "custom_field": "custom_value",  
            "nested": {"key": "value"},  
        },  
    },  
]  
vector_store.add_documents(documents)  
  
# Retrieve with custom metadata  
docs = vector_store.get(documents=[{"id": "1"}])  
print(docs[0].metadata["custom_field"]) # custom_value
```

Common Pitfalls

× Pitfall 1: Collection Name Conflict

```
# Error: Collection already exists  
vector_store = Chroma(  
    # ...  
)
```

```
    collection_name="my_documents",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)

# Fix: Delete existing collection first
vector_store.clear() # Or delete the directory manually
vector_store = Chroma(...) # Now creates fresh
```

× Pitfall 2: Empty Metadata

```
# Error: Empty metadata list
documents = [
    {"page_content": "Document 1"},
    {"page_content": "Document 2"},
]
# metadatas is required for add_texts

# Fix: Provide empty dicts
vector_store.add_texts(
    texts=["Document 1", "Document 2"],
    metadatas=[{}, {}], # Empty dicts, not None
)
```

× Pitfall 3: Mismatched Embeddings

```
# Error: Embedding function mismatch
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=OpenAIEmbeddings(),
    persist_directory="./chroma_db",
)

# If you change embeddings, recreate the store
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=NewEmbeddings(),
    persist_directory="./chroma_db",
)
```

× Pitfall 4: Process Crash on Persistent Chroma

```
# Data is lost if process crashes before index is built
# Fix: Use a larger persist_directory and ensure Chroma has time to index

# Or: Use in-memory for development
```

```
vector_store = Chroma(
    collection_name="my_documents",
    embedding_function=embeddings,
    # No persist_directory for dev
)
```

× Pitfall 5: Wrong Filter Syntax

```
# Error: Invalid filter syntax
results = vector_store.similarity_search(
    "What is Python?",
    filter=["programming"], # Wrong: filters must be dicts

# Fix: Use dict syntax
results = vector_store.similarity_search(
    "What is Python?",
    filter={"category": "programming"},
)
```

Complete Working Example

```
"""
Complete example: Build a semantic search engine with ChromaDB + LangChain
"""

from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.documents import Document

# --- Setup ---
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

# --- Add documents ---
documents = [
    {
        "page_content": "Python is a high-level programming language.",
        "metadata": {"id": "1", "category": "programming", "language":
"Python"},
    },
    {
        "page_content": "JavaScript is a programming language.",
        "metadata": {"id": "2", "category": "programming", "language":
"JavaScript"},
    },
    {
        "page_content": "Machine learning is a subset of AI.",
        "metadata": {"id": "3", "category": "ml", "field": "ai"},
    },
]
```

```
]

# --- Initialize Chroma ---
vector_store = Chroma(
    collection_name="semantic_docs",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)

# --- Add documents ---
vector_store.add_documents(documents)

# --- Search ---
results = vector_store.similarity_search("What is Python?", k=3)

# --- Output ---
for doc in results:
    print(f"--- {doc.metadata['id']} ---")
    print(f"Content: {doc.page_content}")
    print(f"Metadata: {doc.metadata}")
    print()

# --- Delete ---
vector_store.delete(documents=[{"id": "1"}])

# --- Clear ---
# vector_store.clear()
```

License

This documentation is based on the LangChain Chroma integration. Refer to the [LangChain Chroma documentation](#) for the latest updates and official source code.

Generated for LangChain Python ChromaDB integration. Last updated: 2024.